

Notebook

February 8, 2026

1 Атака на широковещательный вариант RSA (RSA Broadcast attack)

Представьте, что Вы используете открытую экспоненту $e = 3$ (не так давно она была достаточно популярна), но вы зашифровываете только сообщения, которые переполняют модуль N при возведении в третью степень, так что нельзя просто взять и извлечь корень третьей степени. Правда ли это безопасно? Есть довольно простой сценарий, который наглядно показывает проблему небольшой открытой экспоненты. Представьте, что пользователь использует RSA, чтобы посылать сообщения на несколько серверов (все с экспонентой 3, но разными модулями N). Пусть таких серверов 3. Одно и то же сообщение шифруется 3 раза: с ключами первого, второго и третьего серверов.

$$C_1 = M^3 \bmod N_1$$

$$C_2 = M^3 \bmod N_2$$

$$C_3 = M^3 \bmod N_3$$

Почти невозможно дешифровать каждый из шифротекстов C_i по одиночке, но с тремя Марвин(или Мэллори, как вам больше нравится) может решить эту задачу.

1.1 Китайская теорема об остатках

Если известны остатки от Евклидова деления целого числа n на несколько целых чисел, то можно определить уникальный остаток от деления n на произведение этих целых чисел, при условии, что все делители (модули, по которым брали остатки) попарно простые.

Как мы можем использовать эту теорему? Во-первых, нам необходимо проверить, что все делители (N_1, N_2, N_3) взаимно (попарно) простые. Но, если вдруг они не взаимно простые, то наибольший общий делитель двух из них больше единицы и либо они равны (и мы не можем их использовать), либо они имеют в составе одно и то же простое число - их . В последнем случае мы и так можем расшифровать сообщение. Так что будем считать, что они попарно взаимнопростые. Это значит, что существует такой X , что:

$$X < N_1 N_2 N_3$$

$$X = C_1 \bmod N_1$$

$$X = C_2 \bmod N_2$$

$$X = C_3 \bmod N_3$$

и такой X уникален.

Давайте рассмотрим $C = M^3$. Так как $M < N_1$ и $M < N_2$ и $M < N_3$, то $C = M^3 < N_1 N_2 N_3$. А ещё:

$$C = C_1 \bmod N_1$$

$$C = C_2 \bmod N_2$$

$$C = C_3 \bmod N_3$$

Так что, используя китайскую теорему об остатках, можно найти C и всё, что останется сделать - это извлечь кубический корень.

1.2 Как получить С?

Пусть $N_i, i = 1, k$ - модули, а $c_i, i = 1, k$ - остатки по делению. $N = N_1 N_2 \dots N_k$, а $M_i = N/N_i$ Тогда $C = (\sum_{i=1}^k C_i M_i (M_i^{-1} \bmod N_i)) \bmod N$

Воспользуйтесь выражением и вытащите флаг из трёх зашифрованных сообщений, полученных с сервера. Удачи!

```
[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1339))
        if show:
            print (self.recv_until().decode())
    def recv_until(self,symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_keys(self,show=True):
        """Receive public keys from the server"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e1=int(re.search('(?!<=e1: )\d+',response).group(0))
```

```

N1=int(re.search('(<=N1: )\d+', response).group(0))
e2=int(re.search('(<=e2: )\d+', response).group(0))
N2=int(re.search('(<=N2: )\d+', response).group(0))
e3=int(re.search('(<=e3: )\d+', response).group(0))
N3=int(re.search('(<=N3: )\d+', response).group(0))

return [(e1,N1),(e2,N2),(e3,N3)]

def get_ciphertexts(self, show=True):
    """Receive ciphertexts from the server"""
    self.s.sendall('ciphertext\n'.encode())
    response=self.recv_until().decode()
    if show:
        print (response)
        c1=bytes_to_long(bytes.fromhex(re.search('(<=ciphertext1:␣
↵)[0-9a-f]+', response).group(0)))
        c2=bytes_to_long(bytes.fromhex(re.search('(<=ciphertext2:␣
↵)[0-9a-f]+', response).group(0)))
        c3=bytes_to_long(bytes.fromhex(re.search('(<=ciphertext3:␣
↵)[0-9a-f]+', response).group(0)))
        return (c1,c2,c3)

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
pk_list=vs.get_public_keys()
(c1,c2,c3)=vs.get_ciphertexts()

```

```

[ ]:

```